

ACP 暑期学校 2026 黑客松

机场过站调度 — 保持墨尔本机场高效运转

(Airport Turnaround Scheduling — Keeping Melbourne Airport Moving)

优化挑战赛 (Optimisation Challenge)

中国长春，2026 年 8 月 24 - 28 日



现在是墨尔本机场（YMLL）的午夜时分。在接下来的二十四小时里，将有数百架飞机降落、停靠、更换机组、加油、装载新一批旅客与行李，然后从各自的停机位推出。它们每一架都需要一个停靠位置，以及一支小型地勤队伍来完成过站作业。做对了，停机坪就像一支排练娴熟的舞蹈；做错了，你就会面对愤怒的旅客、错过的中转、一长队在滑行中白白烧油的飞机，以及代价高昂的一天。

你今天的角色是机场的运行计划员。你的任务就是把一叠航班数据和一本国际航协（IATA）的服务手册，转化为一份机场真正能够执行的计划。数据读写的工具我们已经为你搭好，如何建模和求解这个问题则由你决定。

你要构建什么

一个面向机场运行问题的优化解决方案：给定当天的航班时刻表、停机坪布局、地勤机组排班，以及机位之间的行程矩阵，产出一份计划，为每一个航班分配停机位，为每一项地面保障任务排定时间，并为每项任务配备所需的机组单元。雇用过多的机组就是浪费你的运营资金，但雇用过少则计划不可行。机场需要的是一份既可行又经济的计划。

你可以**使用任何优化技术**，包括 MiniZinc、OR-Tools、Gurobi、手工编写的 SAT 编码，或者纯 Python 的启发式方法。唯一要遵守的就是你读入的 JSON 实例文件和你产出的 JSON 解文件。

祝你好运。机场交给你了。

1 停机位分配

每一架降落的飞机都需要一个停靠位置，而机场的停机位并不可以随意分配。每个机位都有固定的机位大小，位于特定的航站楼，可能只支持客运或货运作业，并且有实际开放的时段。

表 1 给出了一个包含五个真实航班的示例。一个航班可以有不同的承运类型 (Carrier)、机型 (Size) 和运营类型 (Type)。承运类型可以是国内 (domestic) 或者国际 (international)，机型可以是窄体 (narrow) 或者宽体 (wide)，运营类型可以是客运 (passenger) 或者货运 (freighter)。这些属性决定了哪些机位对该航班是可行的。为了对时间进行建模，我们采用从实例窗口起始点开始计数的整分钟时隙 (即每分钟为一个整数单位)。例如，若时间跨度从 00:00 开始，则 `flight_arr=31` 表示 00:31，`flight_arr=754` 表示 12:34，依此类推。

航班	到达时间	离开时间	承运	机型	类型
JST917	00:31	01:16	国内	窄体	客运
UAE9314	12:34	14:49	国际	宽体	客运
JST034	04:02	04:47	国内	窄体	客运
MXD177	04:16	05:01	国内	窄体	客运
CPA3128	14:15	16:30	国内	宽体	货运

表 1: 示例航班及其属性。

表 2 给出了一个包含四个机位的示例供你使用。类似地，一个机位可以有不同的机位大小、用途和支持的类型。机位大小可以是小型 (small) 或者大型 (large)。

停机位	机位大小	用途	支持类型	开放时间	关闭时间
D02	小型	国内	客运	00:00	24:00
D03	大型	国际	客运	00:00	24:00
D04	大型	国际	客运	00:00	24:00
H03	大型	国内	货运	06:00	24:00

表 2: 示例机位及其属性。

我们需要一个可行的停机位分配，满足以下所有条件：

- **C1.** 每个航班都停靠在一个机位，且该航班的整个停留时段（从 `flight_arr` 到 `flight_dep`）必须落在该机位的开放时间 (`open`) 和关闭时间 (`close`) 窗口之内。
- **C2.** 宽体 (wide) 航班需要大型机位 (large)。窄体 (narrow) 航班两种机位大小都能停。
- **C3.** 客运 (passenger) 航班停靠客运机位，货运 (freighter) 航班停靠货运机位。
- **C4.** 国际 (international) 航班必须停靠国际机位，国内 (domestic) 航班必须停靠国内机位。海关可不允许把两者混在一起。

- **C5.** 为了应对不确定性，分配到同一机位的两个航班之间必须至少间隔 **30 分钟** 的过站间隔：如果航班 f_1 在 `flight_dep1` 离开，而航班 f_2 在同一机位于 `flight_arr2` 到达，那么 $\text{flight_arr}_2 - \text{flight_dep}_1 \geq 30$ 分钟。

一旦你得到一个满足 C1–C5 的停机位分配，你就有了一个 **Tier 1** 解。在少量航班上做到可行是容易的，而当我们处理最大实例中数百个航班的排班时，问题就变得有意思了。

2 机组调度

飞机刚刚降落并滑入了它的机位。计时已经开始。在它再次离开之前，必须完成一连串的地面保障任务。旅客下机、行李卸载、飞机加油、客舱清洁、餐食装载、新行李装机、下一批旅客登机、舱门关闭，最后由拖车把飞机推出。有些任务可以并行进行，有些则必须等待。

每项任务需要一支地勤队伍，在任务持续期间在机位现场作业。一项任务需要哪种队伍是由任务本身固定的（例如飞机加油（FUEL）任务总是需要 `fuel_team`）。你来决定由哪个具体的队伍执行每项任务，以及每项任务何时开始。目标是在仍能让每架飞机准时完成过站的前提下取得最小总成本：雇用的机组太少，排班不可行；太多，则是浪费。

一共有七种地勤机组，每种执行一些不同的任务种类：

- `ramp_team`: 飞机到达安全确认 (ARRIVE_SECURE)、旅客下机 (DISEMBARK)、国际文件检查 (INTL_DOCS) 和旅客登机 (BOARDING)。
- `baggage_team`: 行李卸载 (BAG_UNLOAD) 和装载 (BAG_LOAD)、货物卸载 (CARGO_UNLOAD) 和装载 (CARGO_LOAD)。
- `fuel_team`: 飞机加油 (FUEL)。
- `catering_team`: 配餐 (CATERING)。
- `cabin_clean_team`: 客舱清洁 (CABIN_CLEAN)。
- `water_waste_team`: 水/污水 (WATER_LAV)。
- `pushback_team`: 客运航班的推出 (PUSHBACK)，货机的放行 (DEP_RELEASE)。

地勤机组的每支队伍都有用工成本和一个排班窗口。它们的名称在 `LaborID` 数组中给出，而它们的种类、成本和班次窗口则分别在 `labor_kind`、`labor_cost`、`labor_shift_start` 和 `labor_shift_end` 数组中给出。每一种类的队伍有一定的数量的候选队伍，但否真正雇用它们由你决定。

每个航班任务列表由其任务档案 (*service profile*) 固定，即承运类型、机型和运营类型的组合。任务档案决定了该航班需要哪些任务以及它们之间的先后关系。一共有三种结构上不同的任务图，每个档案族对应一种。它们分别绘制在图 1、2 和 3 中。每个任务节点按其所需机组的人力种类进行颜色编码：`ramp_team` `baggage_team` `fuel_team` `catering_team` `cabin_clean_team` `water_waste_team` `pushback_team`。节点上还标注了以分钟为单位的时长范围，形式为窄体 / 宽体。每项任务的时长在不同航班之间可能略有差异，但始终保持在指定范围之内。

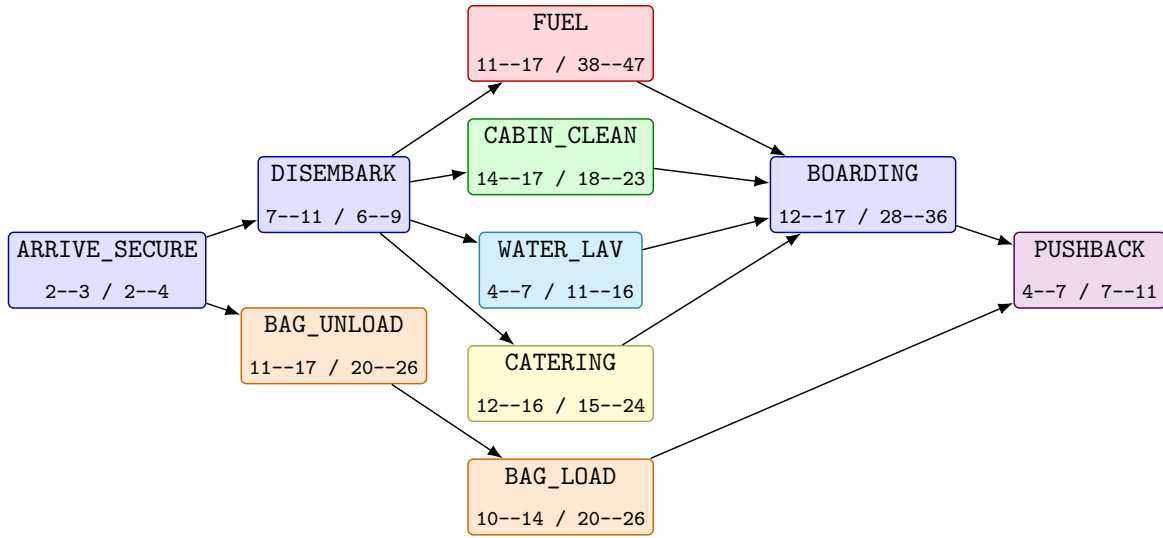


图 1: 国内客运地勤任务 (Domestic passenger profile)。时长以分钟为单位。

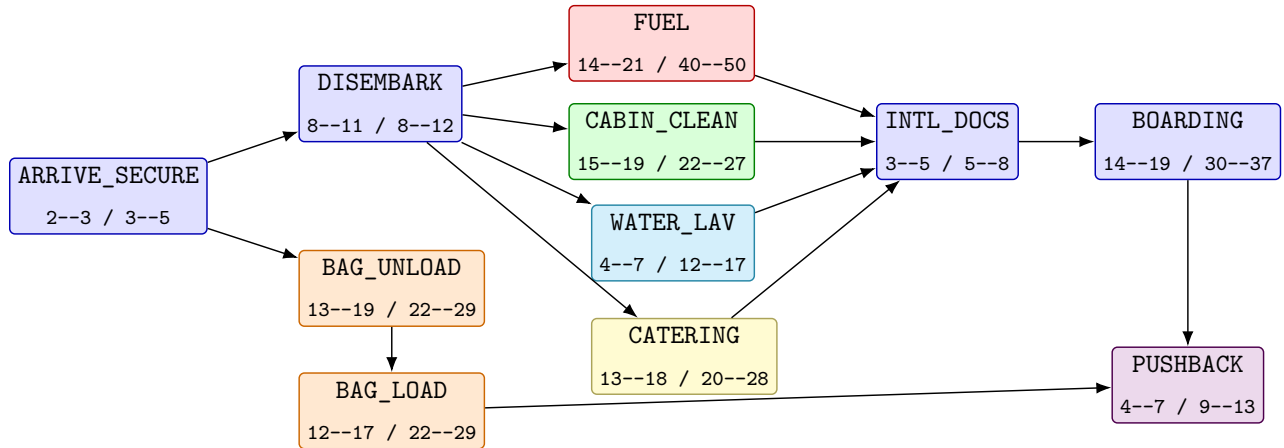


图 2: 国际客运地勤任务 (International passenger profile)。时长以分钟为单位。

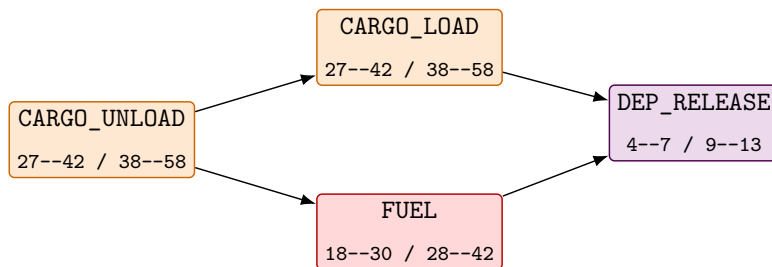


图 3: 货机地勤任务 (Freighter profile)。时长以分钟为单位。

现在，对于每个航班，我们还需额外决定每项任务的开始时间以及由哪个具体队伍执行它。在仍然必须成立的 C1–C5 之上，开始时间与队伍分配还必须满足：

- **C6.** 你分配给某项任务的队伍必须是该任务所需的种类。
- **C7.** 每项任务在航班到达时刻或之后开始，并在其离开时刻或之前结束。
- **C8.** 只要一个航班的两项任务在种类上存在先后关系（见图 1、2 和 3），前面的任务必须在后续任务开始之前结束。
- **C9.** 一支队伍一次只做一项任务：任何队伍都不得同时被安排在两项时间上重叠的任务上，无论是在同一航班还是不同航班上。
- **C10.** 每支队伍只在自己的 `shift` 窗口内工作，即从 `labor_shift_start` 中的开始时间到 `labor_shift_end` 中的结束时间：它执行的每项任务都必须在该窗口内开始并结束。

我们的目标是最小总成本：你计划中出现至少一次的每支队伍的用工成本之和。注意，不同班次的队伍成本可以不同。

这是一个真正的**机组调度**问题：你要选择每项任务的开始时间（在其窗口和先后关系之内）以及配置哪些班次，使得同种类的并发任务数量永远不超过你雇用的队伍数。一旦 C1–C10 成立，你就有了一个 **Tier 2** 解。

3 附加内容：路程与休息

这份草拟的计划在纸面上看起来很完美，但你几乎立刻可以注意到两件事。第一，同一辆加油车单元在机位 B22 完成作业后，可能在一分钟之后就要在 H03 开始作业，但车辆穿越机场是需要时间的。第二，一支刚刚连续用 90 分钟完成了三次过站的坡道队伍，在下一次作业前需要休息一下。

假设每个机组单元在停机坪上都以相同的速度移动，且任意两个机位之间的行程时间是已知的。现在，让我们把每个具体队伍的时间线在物理上变得真实。此时又有两条约束发挥作用：

- **C11.** 一支队伍必须有足够的时间在分配给它的连续两项任务的机位之间行进。当然，如果两项任务分配在同一机位，行程时间为零。
- **C12.** 一支队伍不能长时间连续工作而不休息。按时间顺序遍历一支队伍的任务，并追踪其当前连续工作时间（work streak）的起点（第一项任务即开启一个连续段）。这个连续工作时间会跨越短暂的间隔以及行程而延续，只有当队伍在两项连续任务之间获得至少 15 分钟的休息时才会重置。一支队伍开始某项任务的时刻，不得比其当前连续工作的起点晚 90 分钟以上。注意，在停机坪之间移动不算作休息，空闲时间等于两项任务之间的间隔减去在两个机位之间行进所需的时间。例如，一支已经连续工作 80 分钟，随后又花 15 分钟行进到下一个机位的队伍，仍然没有得到任何休息，因而不能立即开始一项任务。

数据中附带一个以机位为下标的对称整数矩阵 `travel`，其中 `travel[g1][g2]` 给出把一个单元从机位 `g1` 移动到机位 `g2` 所需的分钟数。同机位之间的转换为零。对于分配给同一单元的任意两项连续任务 (f_1, t_1) 和 (f_2, t_2) ，若 `task_end[f1, t1] ≤ task_start[f2, t2]`，则其间隔必须覆盖 f_1 与 f_2 停靠机位之间的行程：

$$\text{task_start}[f_2, t_2] \geq \text{task_end}[f_1, t_1] + \text{travel}[\text{gate}[f_1], \text{gate}[f_2]].$$

当两个机位分处机场两端时，这可能成为迫使你多雇队伍以完成全部任务的决定性因素。

满足全部约束 C1–C12 就得到一个 **Tier 3** 解。相较于仅满足 C1–C9 所允许的情况，约束 C11 和 C12 通常会把总雇用成本推高：一支此前本可以做完所有任务的队伍，可能现在需要行程余量或休息。因此你可能需要多雇一个该种类的单元来消化这部分余量。

4 数据格式

实例和提交都采用 JSON 格式。下面展示一个包含五个航班和四个机位的数据实例。

Listing 1: Instance JSON

```
{
  "horizon": 1440,
  "FlightID": ["JST917", "UAE9314", "JST034", "MXD177", "CPA3128"],
  "GateID": ["D02", "D03", "D04", "H03"],
  "LaborID": ["ramp_team_01", "ramp_team_02", "baggage_team_01",
    /* ... one per team unit ... */],
  "max_tasks": 11,
  /* Data for flights */
  "flight_arr": [31, 754, 242, 256, 855],
  "flight_dep": [76, 889, 287, 301, 990],
  "flight_carrier": ["domestic","international","domestic",
    "domestic","domestic"],
  "flight_size": ["narrow","wide","narrow","narrow","wide"],
  "flight_op_type": ["passenger","passenger","passenger",
    "passenger","freighter"],
  "flight_n_tasks": [10, 11, 10, 10, 4],
  "flight_tasks": [
    [ {"kind":"ARRIVE_SECURE","duration":2},
      {"kind":"DISEMBARK","duration":7},
      /* ... flight_n_tasks[0] real tasks, then padded to max_tasks ... */
      {"kind":"PUSHBACK","duration":0} ],
    /* ... one padded row per flight ... */
  ],
  /* Data for gates */
  "gate_stand_size": ["small","large","large","large"],
  "gate_usage": ["domestic","international","international","domestic"],
  "gate_op_type": ["passenger","passenger", "passenger","freighter"],
  "gate_open": [0, 0, 0, 360],
  "gate_close": [1440, 1440, 1440, 1440],
  /* Data for labor units */
  "labor_kind": ["ramp_team","ramp_team","baggage_team", ...],
  "labor_cost": [220, 231, 219, ...],
  "labor_shift_start": [0, 104, 104, ...],
  "labor_shift_end": [209, 313, 313, ...],
  /* Travel time matrix (|GateID| x |GateID| = 4x4) */
  "travel": [[0, 2, 3, 9], [2, 0, 1, 10], [3, 1, 0, 11], [9, 10, 11, 0]]
}
```

实例 JSON 包含若干数据：

- **horizon** 是实例窗口的长度，以 1 分钟为一个时隙。
- **FlightID**、**GateID** 和 **LaborID** 是有序的标识符数组，供下面各数组使用。这些数组的顺序是固定的，你的输出必须遵守它。
- 航班有若干属性：
 - **flight_arr/flight_dep** 给出时间窗口。
 - **flight_carrier/flight_size/flight_op_type** 给出服务档案。
 - **flight_n_tasks** 给出真实任务的数量。
 - **flight_tasks** 给出有序的任务列表，每一项为 {**kind**, **duration**}。它是一个补齐到 **max_tasks** 列的矩形数组，每行只有前 **flight_n_tasks** 项是真实的；补齐项（时长为 0）必须忽略。
- 机位属性有若干数组：
 - **gate_stand_size**、**gate_usage**、**gate_op_type** 分别表示机位的大小、用途和支持类型。
 - 每个机位有一个开放窗口 [**gate_open**, **gate_close**]。
- 地勤机组的队伍有若干属性：地勤机组类型 **labor_kind**，雇用成本 **labor_cost**，以及它可以工作的班次窗口 [**labor_shift_start**, **labor_shift_end**]。
- **travel** 是一个 $|\text{GateID}| \times |\text{GateID}|$ 的对称矩阵，各项为非负、以 1 分钟为时隙（对应穿越指廊/停机坪布局的走廊行程）。

提交格式 一份提交答案是一个包含四组数据的 JSON 对象。每项任务恰好由一支队伍完成，因此 **task_labor[f][t]** 是单个 LaborID。

Listing 2: Example solution submission

```
{
  "gate":      ["D02", "D03", "D02", "D02", "H03"],
  "task_start": [[31, 33, 33, 40, 40, 40, 40, 44, 54, 66, 0],
                 /* ... one row per flight, padded to max_tasks ... */],
  "task_labor": [ ["ramp_team_01", "ramp_team_01",
                  "baggage_team_01", "fuel_team_01",
                  /* ... one LaborID per task, padded ... */],
                 /* ... one row per flight ... */],
  "cost": 4231
}
```

- **gate**: 大小为 $|\text{FlightID}|$ 的数组，为每个航班保存所选的 GateID。

- **task_start**: 大小为 $|\text{FlightID}| \times \text{max_tasks}$ 的二维数组, 保存每项任务的开始时隙 (1 分钟)。超出某航班 **flight_n_tasks** 的补齐项会被忽略, 可设为零。
- **task_labor**: 大小为 $|\text{FlightID}| \times \text{max_tasks}$ 的二维数组; 每一项是你分配给该任务的单个 **LaborID**。补齐项可以是任意合法的 **LaborID**。
- **cost**: 计划中任何地方出现过的、各不相同的人力单元的成本之和。

在本例中, 第一个航班的每项任务都不早于 31 开始、不晚于 76 结束, 先后关系图得到遵守, 且每项任务都由所需种类的队伍完成。评分器会按所使用的、各不相同的 **LaborID** 重新计算成本, 并检查它是否与所报告的 **cost** 相符。

5 竞赛与排名

本次竞赛中有多个实例。每个实例独立评分，你的总分是各实例得分之和。排行榜按此总分对参赛队伍进行排名。

单个实例上的分数按三个层级（Tier）授予。Tier 1 是一旦满足停机位可行性约束即给予的固定奖励；Tier 2 和 Tier 3 追加的分数则随你的计划相对于排行榜当前最佳成绩有多便宜而按比例变化。表 3 概括了评分方案。

层级	约束	分数公式	每实例最高分
1	C1-C5	20	20
2	C1-C10	$50 \times \min(1, \text{best_tier2_cost}/\text{cost})$	50
3	C1-C12	$30 \times \min(1, \text{best_tier3_cost}/\text{cost})$	30

表 3: 每实例分数方案。

设 `best_tier2_cost` 和 `best_tier3_cost` 分别是本实例排行榜上迄今为止见到的 Tier 2 和 Tier 3 解的最佳成本。如果你的提交满足 C1-C10 并报告成本 `cost`，你会获得 Tier 1 的 20 分，加上 Tier 2 最多 50 分，按你的成本与迄今最佳的接近程度进行缩放。如果你的解进一步满足 C11 和 C12，你还可以为 Tier 3 赚取最多 30 分，同样按迄今最佳 Tier 3 成本进行缩放。

算例说明。 在某个中等规模实例上，排行榜当前显示 Tier-2 最佳为 220，Tier-3 最佳为 240。你提交了一份满足所有约束的计划，报告成本为 245。如果该成本与计划决策相符且 C1-C12 全部成立，则你在该实例上的分数为

$$20 + 50 \times \min\left(1, \frac{220}{245}\right) + 30 \times \min\left(1, \frac{240}{245}\right) \approx 20 + 44.9 + 29.4 = 94.3 \text{ 分}。$$

同样这份成本为 245 的计划。如果提交的答案违反了某个 Tier-3 约束（C11 或 C12 之一），则只按 Tier 2 评级，在该实例上得 $20 + 44.9 = 64.9$ 分。一份只满足 Tier-1 的解，无论成本多少，都固定得 20 分。

提交。 你可以在 8 月 28 号 16:00 之前通过竞赛网站提交任意次数。网站为每个实例保留你的最佳解。排行榜将在竞赛截止前半小时隐藏，最终评分以你所做的最佳提交为准。